

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Software Engineering Laboratory — Lab Report

Name	Suhas D
SRN	PES1UG24AM451
Programme	B.Tech, Computer Science & Engineering (AI & ML)
Institution	PES University, Bengaluru
Jira Project	Feature Flag Manager (Key: SCRUM) — Company-managed Scrum
Submission Date	06 September 2026

1. Objective

To convert the functional requirements identified in Lab 1 into an Agile backlog in Jira — structured as Epics and User Stories — to prioritise and estimate that backlog using the Fibonacci story-point scale, to simulate two one-week Sprints by progressing work items across the Scrum board, and to analyse sprint progress using Jira's Burndown Chart.

2. Project Setup

A company-managed **Scrum** project named **Feature Flag Manager** (project key **SCRUM**) was created in Jira. The scenario carried forward from Lab 1 is a **feature-flag management service** — a system that lets a development team enable, disable and progressively roll out application features at runtime without redeploying code, with per-environment overrides and client SDK synchronisation.

The functional requirements from Lab 1 were grouped into three functional themes, which became the three Epics of the project. Five User Stories were written under these Epics, prioritised, estimated with story points, and executed across two one-week Sprints.

3. Backlog: Epics and User Stories

The backlog was structured with each User Story linked to a parent Epic, so that the Epic panel in Jira groups related stories together. User Stories follow the standard Agile format — *As a <role>, I want <goal>, so that <benefit>*.

Work Item	Summary	User Story (As a / I want / So that)
EPIC 1 (SCRUM-5) — Feature Flag Management • Enable software engineers to create and manage feature flags with unique keys and default states.		
SCRUM-8	Create Feature Flag	As a developer, I want to create a named feature flag with a description and a default on/off state, so that I can control a new feature at runtime without redeploying the application.
EPIC 2 (SCRUM-6) — Feature Flag Configuration • Enable release managers to configure feature flag evaluation, percentage-based rollouts, and environment-specific overrides.		
SCRUM-9	Evaluate Feature Flag	As an application service, I want to evaluate a feature flag for a given user or context and receive its resolved value, so that the correct behaviour is served to each request.
SCRUM-10	Configure Rollout Percentage	As a release manager, I want to enable a feature for a defined percentage of users, so that I can roll a feature out gradually and limit the blast radius of a defect.
SCRUM-11	Set Environment Override	As a developer, I want to override a flag's value for a specific environment (dev / staging / production), so that a feature can be tested in lower environments before it is exposed in production.
EPIC 3 (SCRUM-7) — Client Synchronization • Synchronize feature flag configuration changes with connected client SDKs.		

SCRUM-12	Synchronize Client SDK	As a client application, I want to receive up-to-date flag configuration from the service and cache it locally, so that flag changes take effect quickly without a restart and evaluation still works if the service is briefly unreachable.
----------	------------------------	---

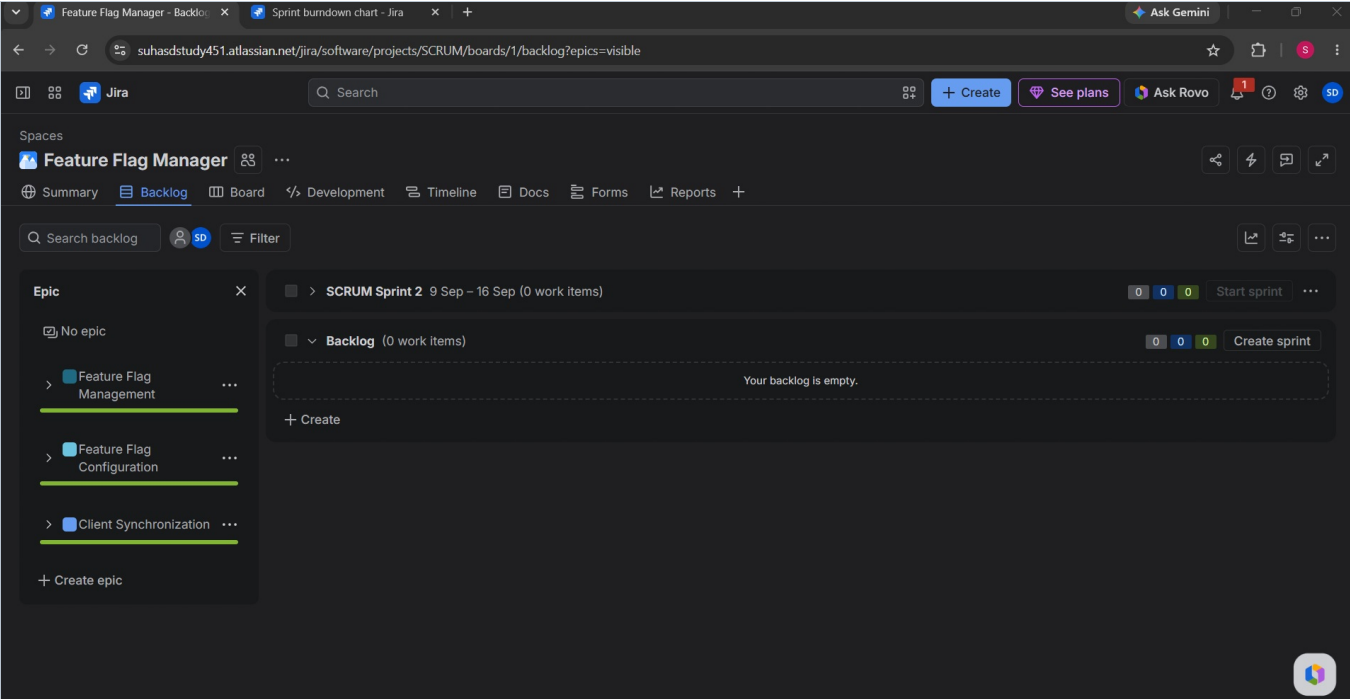


Figure 1 — Jira Backlog view with the Epic panel enabled, showing the three Epics of the project: Feature Flag Management, Feature Flag Configuration and Client Synchronization. The completion bar under each Epic is fully green, and the backlog is empty because every work item has been pulled into a sprint and completed.

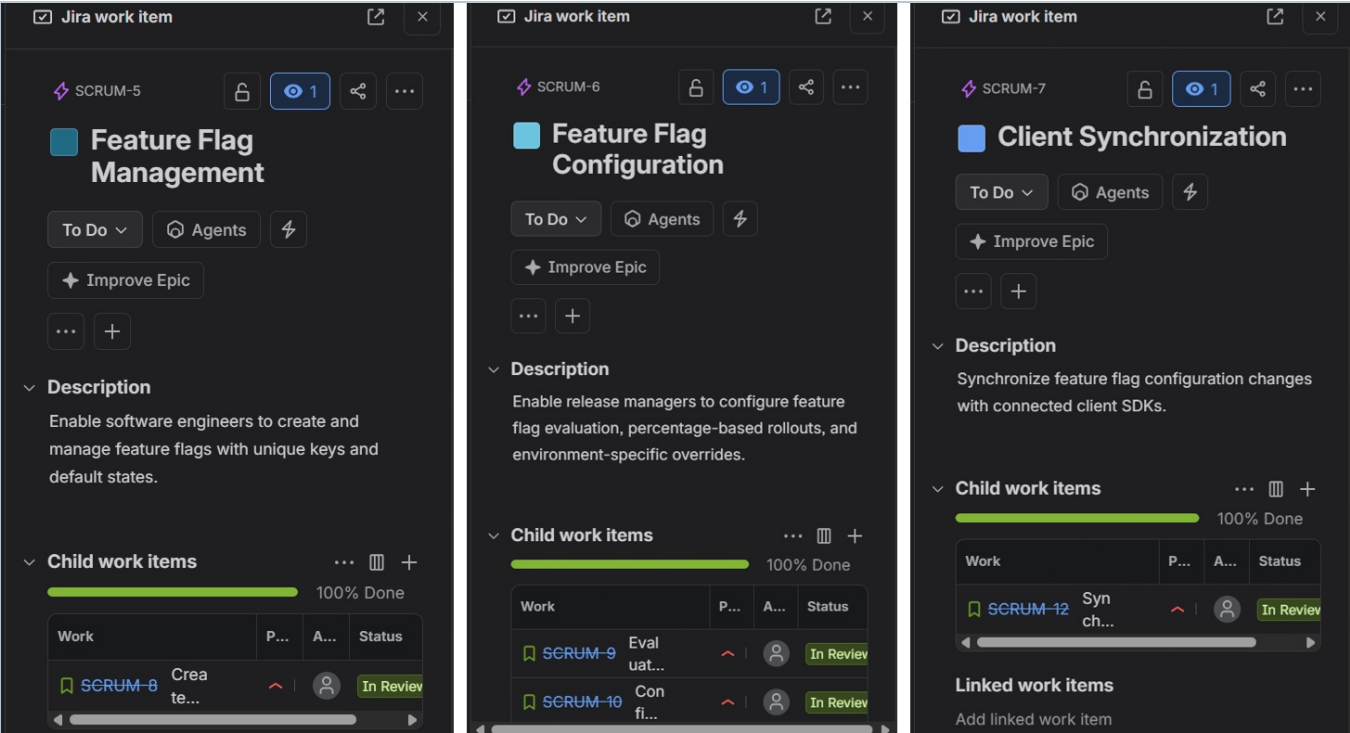


Figure 2 — Epic detail panels for SCRUM-5, SCRUM-6 and SCRUM-7, showing each Epic's description and its child work items. All three Epics report **100% Done**: SCRUM-8 under Feature Flag Management; SCRUM-9, SCRUM-10 and SCRUM-11 under Feature Flag Configuration; and SCRUM-12 under Client Synchronization.

4. Prioritisation and Story Point Assignment

Each User Story was assigned a **priority** (based on user impact and dependency order) and a **story point** estimate on the Fibonacci scale (1, 2, 3, 5, 8, 13...), reflecting relative complexity, volume of work and uncertainty rather than clock hours.

Item	Summary	Priority	Points	Sprint	Basis for the estimate
SCRUM-8	Create Feature Flag	High	3	Sprint 1	Well-understood CRUD work over a single entity; low uncertainty.
SCRUM-9	Evaluate Feature Flag	High	5	Sprint 1	Rule resolution logic plus context handling; must be correct and fast on every request.
SCRUM-10	Configure Rollout Percentage	High	5	Sprint 1	Requires stable, consistent user bucketing (hashing) so a user does not flip between variants.
SCRUM-11	Set Environment Override	Medium	3	Sprint 2	An extension of existing configuration; mostly precedence rules over work already built.
SCRUM-12	Synchronize Client SDK	High	8	Sprint 2	Largest and least certain: caching, refresh strategy, stale-data and network-failure handling.
Total committed			24	13 pts (Sprint 1) + 11 pts (Sprint 2)	

Prioritisation rationale: the backlog was ordered so that dependencies resolve first — a flag must exist before it can be evaluated, and evaluation must work before a percentage rollout is meaningful. *Set Environment Override* was rated Medium because it refines behaviour that already works rather than unblocking anything, and was therefore deferred to Sprint 2.

5. Sprint 1 — Execution

Sprint	SCRUM Sprint 1
Duration	02 September 2026 - 09 September 2026 (1 week)
Sprint goal	Implement core feature flag creation, evaluation, and rollout functionality.
Committed	3 work items — 13 story points
Completed	3 work items — 13 story points (100%)

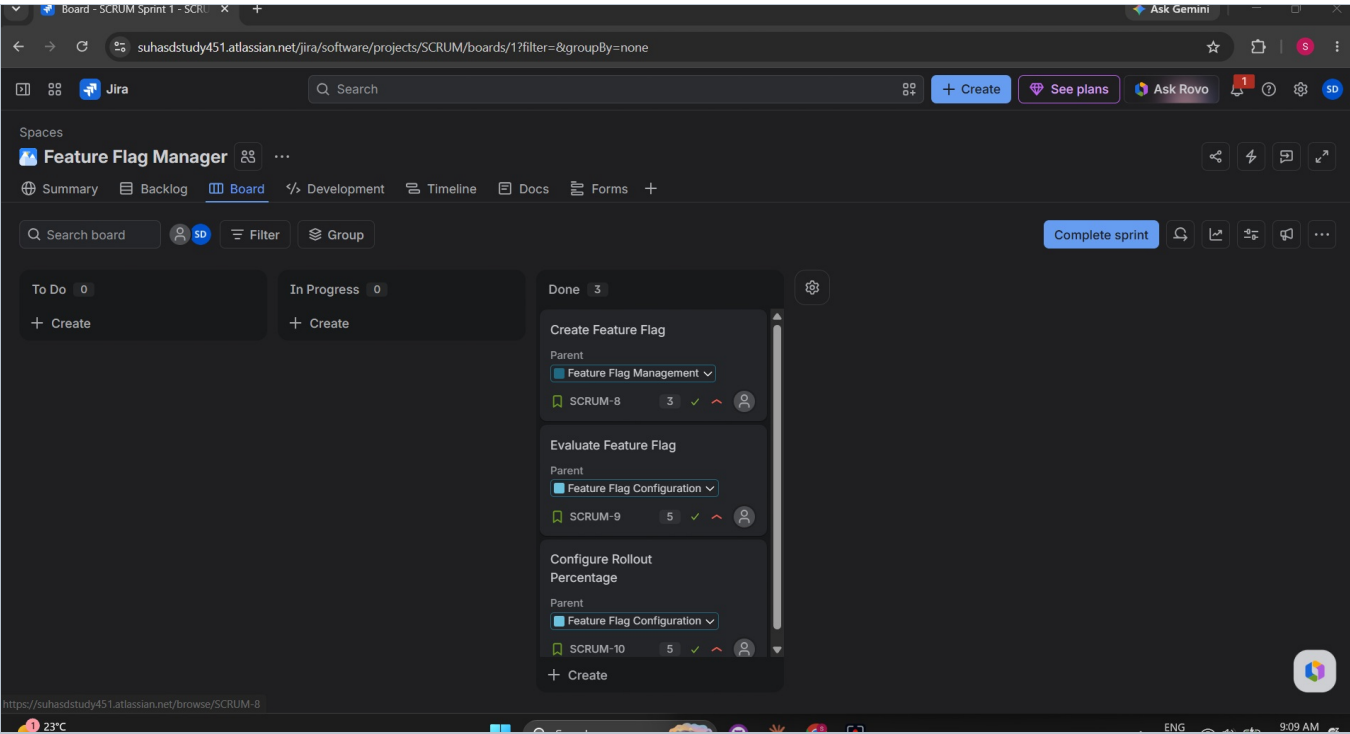


Figure 3 — SCRUM Sprint 1, Active Sprint board. All three committed stories (SCRUM-8, SCRUM-9, SCRUM-10) have been progressed To Do → In Progress → Done; To Do and In Progress are empty at sprint close.

6. Sprint 2 — Execution

Sprint	SCRUM Sprint 2
Duration	06 September 2026 - 13 September 2026 (1 week)
Sprint goal	Complete environment override and client SDK synchronization functionality.
Committed	2 work items — 11 story points
Completed	2 work items — 11 story points (100%)

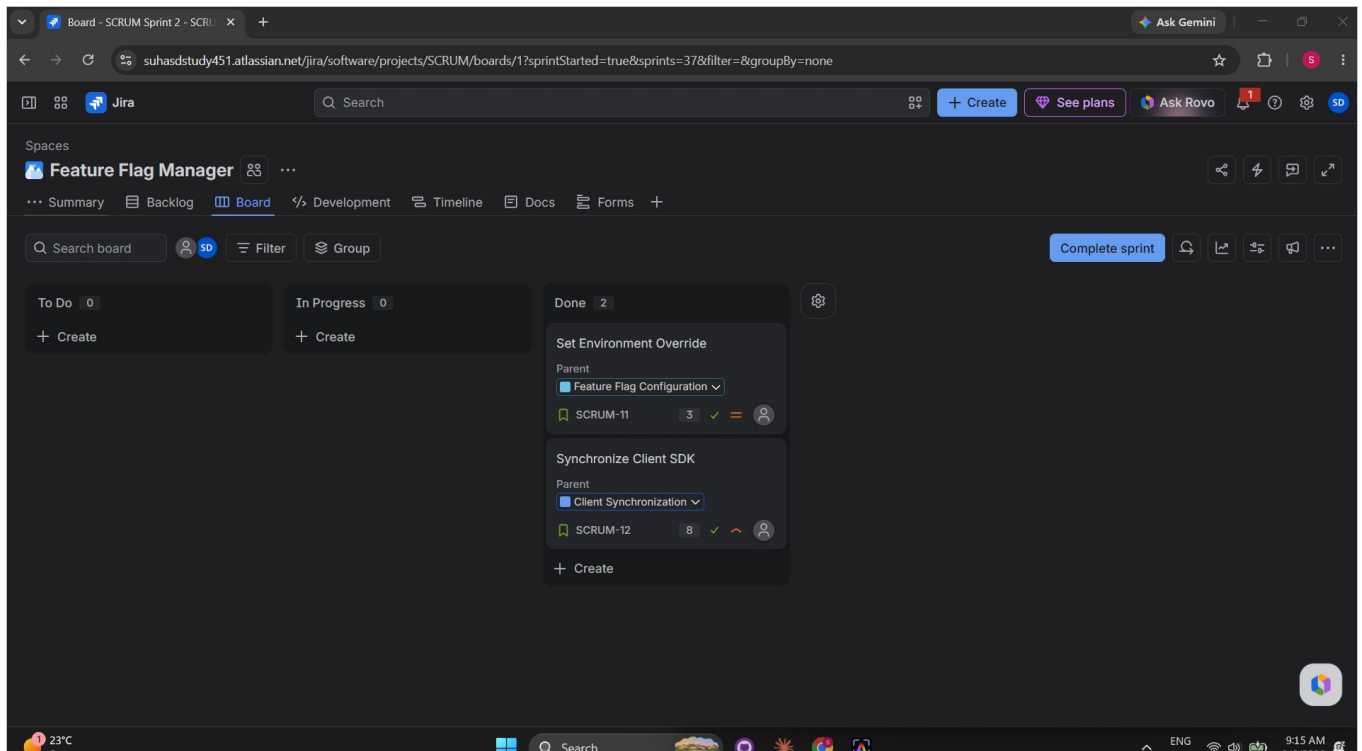


Figure 4 — SCRUM Sprint 2, Active Sprint board. SCRUM-11 (3 pts) and SCRUM-12 (8 pts) completed; the sprint carried no items forward.

7. Burndown Charts

The burndown chart plots **remaining work** (red) against the **guideline** (grey) — the ideal linear burn rate from the committed total down to zero across the sprint. A red line below the guideline indicates the team is ahead of the ideal rate; above it indicates the team is behind.

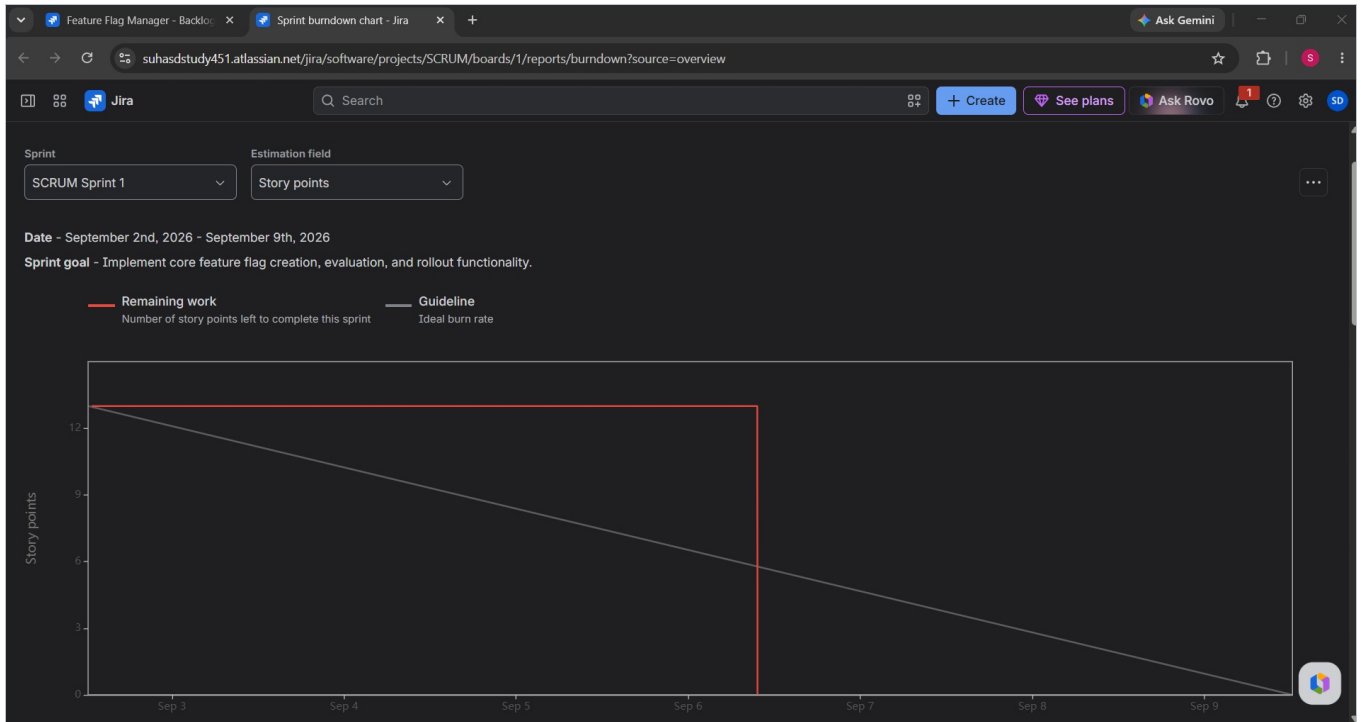


Figure 5 — Sprint 1 Burndown Chart. Remaining work holds flat at 13 story points from 02 September to 06 September, then falls vertically to 0 in a single step.

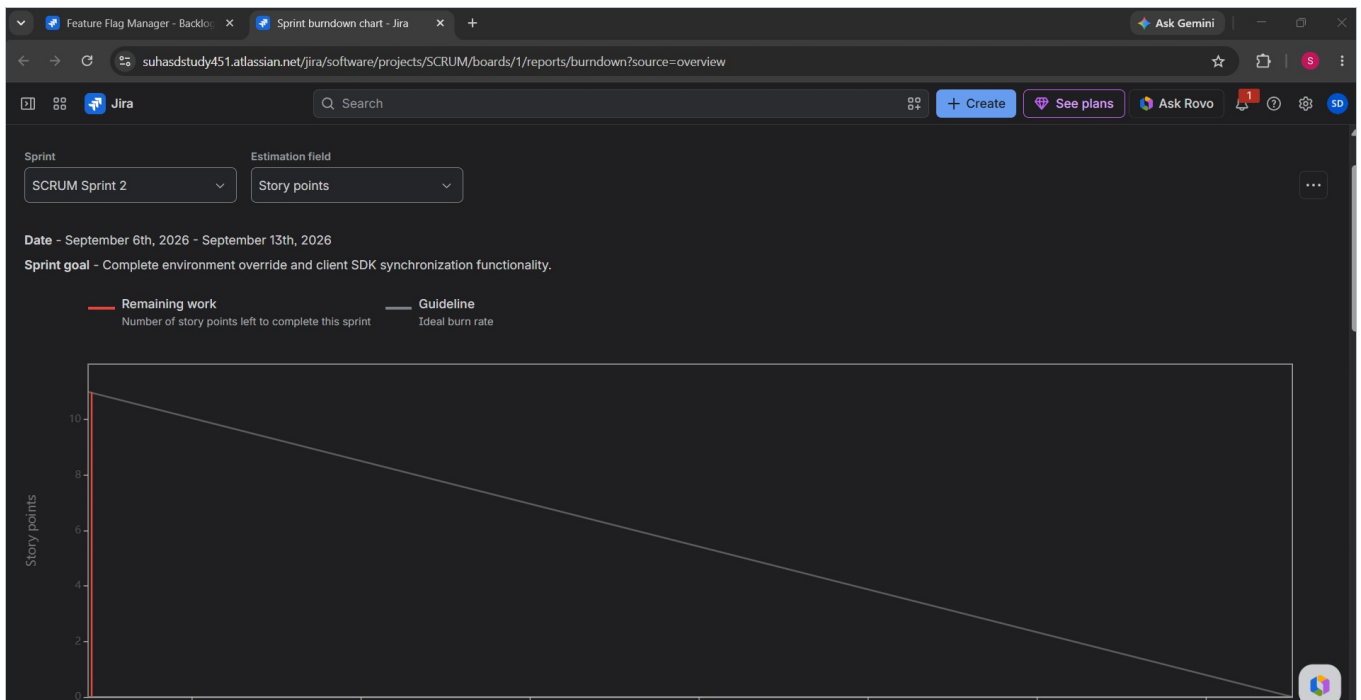


Figure 6 — Sprint 2 Burndown Chart. All 11 story points are burnt down in a near-vertical drop at the start of the sprint, leaving the remaining-work line at zero well below the guideline.

8. Reflection Questions

1. Did your estimations reflect the actual effort?

At the sprint level, yes — both sprints closed with 100% of the committed points completed and nothing carried forward, so no sprint was over-committed. The *relative* ordering of the estimates also held up: *Create Feature Flag* (3) was genuinely the most routine item, while *Synchronize Client SDK* (8) absorbed the most effort because caching, refresh strategy and failure handling each introduced decisions that were not visible when the story was first written. The 5-point items sat correctly between the two.

Being honest about the limits of that conclusion: the burndown charts show no incremental burn, so there is no day-by-day evidence to validate the estimates against — only the endpoint. Two further factors weaken the check. First, the estimates were produced by a single estimator rather than through a real planning-poker spread, so there was no disagreement to expose hidden assumptions; an estimate that is never challenged tends to be anchored on the first number considered. Second, estimate quality in Scrum is only demonstrable over several sprints, once velocity stabilises. Two sprints of 13 and 11 points is a data point, not a trend.

2. Was your backlog well-prioritized?

Largely yes, on dependency grounds. The three Sprint 1 items were ordered so that each unblocked the next — a flag cannot be evaluated before it can be created, and a percentage rollout is meaningless before evaluation works. *Set Environment Override* was correctly rated Medium and deferred: it refines configuration that already functions rather than unblocking anything downstream. *Synchronize Client SDK* was rated High in Sprint 2 because it is the mechanism by which every preceding feature actually reaches end users; without it the service is complete but has no consumer.

The weakness is that priority tracked the dependency chain almost exactly, which means the backlog was ordered by *technical sequence* rather than by *business value*. Those two orderings happened to coincide here, but they often diverge — with a real product owner, environment overrides might well have outranked percentage rollout because developers need them from day one. A better-prioritised backlog would show at least some points where value pulled against dependency order and a deliberate trade-off was made.

3. How did your simulated sprint align with your plan?

Alignment was complete in the narrow sense: Sprint 1 delivered 3 of 3 stories (13 of 13 points) and Sprint 2 delivered 2 of 2 stories (11 of 11 points), with no scope added or removed mid-sprint and no carry-over between sprints. Both sprint goals were met as written.

Two deviations from real Scrum practice should be recorded. Sprint 1 was closed on 06 September against a planned end date of 09 September, and Sprint 2 was started the same day — so the two sprints overlap on the calendar, which would not happen in a genuine cadence where one sprint closes before the next is planned. This is an artefact of compressing a two-sprint simulation into a single lab session. More substantively, a plan that matches execution perfectly is not by itself a sign of good planning: in real sprints, discovery during implementation almost always forces some re-scoping, and the absence of any deviation here suggests the stories were executed as a checklist rather than genuinely elaborated during the sprint.

4. What insights did the burndown chart give about your team's capacity?

The chart shape is the most informative part of the result. Sprint 1's remaining-work line stays flat at 13 points for four of the seven days and then drops vertically to zero; Sprint 2's line drops almost immediately at the sprint start. Neither is a burn — both are **batch completion**, produced because every story was moved To Do → In Progress → Done in one sitting rather than closed as work genuinely finished. Three insights follow:

- **Measured throughput is not sustainable capacity.** The two sprints yield 13 and 11 points, averaging roughly 12 points per one-week sprint. That figure is a completion total for a simulated sprint, not an empirically observed velocity, and it should not be used to forecast a third sprint.
- **A flat line followed by a cliff is a warning sign in a real sprint.** It means work sits in progress with nothing closing, so the team learns whether it will hit the goal only at the very end — precisely when there is no time left to descope or ask for help. The value of a burndown is early warning, and this shape destroys it.
- **Story size drives whether the chart is usable at all.** For a burndown to show a meaningful slope, stories must be small enough that at least one closes most days. On a one-week sprint the 8-point *Synchronize Client SDK* story is the clear candidate for splitting — for example, into initial configuration fetch, background refresh, and offline/stale-cache fallback — which would produce three closable increments instead of one large one.

The practical lesson is that a burndown chart is only as useful as the discipline of updating item status as work actually progresses; a chart maintained retrospectively reports history but provides no control.

9. Conclusion

The Lab 1 functional requirements for the Feature Flag Manager were successfully converted into a structured Agile backlog of 3 Epics and 5 User Stories in Jira, prioritised by user impact and dependency, and estimated at 24 story points on the Fibonacci scale. Two one-week Sprints were planned and executed to completion (13 and 11 points), with all work items progressed across the Scrum board from To Do to Done, and sprint progress analysed using Jira's Burndown Chart. The exercise demonstrated not only the mechanics of Epics, story points, sprint boards and burndown reports, but also — through the shape of the burndown itself — why incremental status updates and appropriately sized stories are what make Agile progress tracking meaningful.